

Week 2: Product and Category Management System

Scenario:

The e-commerce company wants to maintain information about products and their categories. Each product belongs to a specific category, and the system should manage product details such as name, price, and stock availability.

1. Design Product and Category Tables

Category Table:

```
SQL> CREATE TABLE CATEGORY(CATEGORY_ID VARCHAR(7),CATEGORY_NAME VARCHAR(20),DESCRIPTION VARCHAR(50));  
Table created.
```

Product Table:

```
SQL> CREATE TABLE PRODUCT(PRODUCT_ID VARCHAR(7),PRODUCT_NAME VARCHAR(20),CATEGORY_ID VARCHAR(7),PRICE NUMBER(9,2),STOCK_QUANTITY NUMBER);  
Table created.
```

The two tables for Category and Product have been created.

Now, we have to find the relationship between the tables and apply constraints to the tables.

2. Define Relationships and Constraints

Relationships:

One Category Can have many Products.

The relationship between Category and Product is One to many (1-> *).

We found that, what is their relationship. Now we have to discuss that which is the key that is going to make this relationship.

Before that, we need to understand about KEY Constraints.

PRIMARY KEY:

The Column(s) or Field(s) that uniquely identifies a row / tuple

In these two tables, which field can be a Primary Key?

In Category Table, Category_ID is a unique Value that helps to identify each category. Likewise, in Product Table Product_ID is a unique Column.

FOREIGN KEY:

The Key which links two tables.

Therefore, Category_ID acts as Foreign Key in the Product Table.

We can explore some other constraints like NOT NULL, CHECK and UNIQUE

NOT NULL:

Ensures that the Column cannot have a empty (null) value.

For Example, a Product name or Category name cannot be empty. All category or product must have a name.

CHECK:

Validates the values in a column satisfies the condition.

For Example, To check the Price of a Product must always be a positive number.

UNIQUE:

Ensures that there is no repeated value for a column.

For Example, If we mention UNIQUE Constraint to Category Name, no two category can have same name.

Defining Relationship and Creating Constraints for Product and Category:

Primary Key:

```
SQL> ALTER TABLE CATEGORY ADD CONSTRAINT PK_CATEGORY PRIMARY KEY (CATEGORY_ID);  
Table altered.  
  
SQL> ALTER TABLE PRODUCT ADD CONSTRAINT PK_PRODUCT PRIMARY KEY (PRODUCT_ID);  
Table altered.
```

Creating Relationship using FOREIGN KEY:

```
SQL> ALTER TABLE Product ADD CONSTRAINT FK_Product_Category FOREIGN KEY (Category_ID) REFERENCES Category(Category_id);  
Table altered.
```

Adding Constraints:

NOT NULL:

```
SQL> ALTER TABLE Category MODIFY Category_Name VARCHAR(100) NOT NULL;  
Table altered.
```

```
SQL> ALTER TABLE Product MODIFY Product_Name VARCHAR(150) NOT NULL;  
Table altered.
```

CHECK:

```
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_PRICE CHECK (Price > 0);
Table altered.
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_Stock CHECK(Stock_Quantity >= 0);
Table altered.
```

UNIQUE:

```
SQL> ALTER TABLE Category ADD CONSTRAINT UQ_Category UNIQUE (Category_Name);
Table altered.
```

3. Insert Values:

Category Table:

```
SQL> insert into Category values('C1001','Electronics','Electronic devices and accessories');
1 row created.
```

```
SQL> insert into Category values('C1002','Clothing','Mens and womens clothing');
1 row created.
```

```
SQL> insert into Category values('C1003','Groceries','Food and Daily-use grocery items');
1 row created.
```

```
SQL> insert into Category values('C1004','Furniture','Home and Office furniture');
1 row created.
```

```
SQL> insert into Category values('C1005','Books','Educational and general books');
1 row created.
```

After Insertion Category Table:

```
CATEGOR CATEGORY_NAME
-----
DESCRIPTION
-----
C1001   Electronics
Electronic devices and accessories

C1002   Clothing
Mens and womens clothing

C1003   Groceries
Food and Daily-use grocery items

C1004   Furniture
Home and Office furniture

C1005   Books
Educational and general books
```

Product Table:

```
SQL> insert into product values('P1001','Laptop','C1001',55000,20);
1 row created.

SQL> insert into product values('P1002','T-shirt','C1002',799,50);
1 row created.

SQL> insert into product values('P1003','Rice bag','C1003',1200,40);
1 row created.

SQL> insert into product values('P1004','Office chair','C1004',4500,15);
1 row created.

SQL> insert into product values('P1005','Python book','C1005',650,25);
1 row created.
```

After Insertion Product Table:

PRODUCT		

PRODUCT_NAME		

CATEGOR	PRICE	STOCK_QUANTITY
-----	-----	-----
P1001		
Laptop		
C1001	55000	20
P1002		
T-shirt		
C1002	799	50
P1003		
Rice bag		
C1003	1200	40
P1004		
Office chair		
C1004	4500	15
P1005		
Python book		
C1005	650	25

CRUD Operations:

Insert Record:

```
SQL> insert into product values('P1006','Headphones','C1001',1500,30);
1 row created.
```

Update Price:

```
SQL> update product set price = 2000 where product_name = 'Rice bag';
1 row updated.
```

Delete a Product:

```
SQL> delete from product where product_name = 'Laptop';  
1 row deleted.
```

5. Generate Category-wise Product Reports

Write SQL queries to:

- Display all products under each category.

CATEGORY_NAME		

PRODUCT_NAME		

	PRICE	STOCK_QUANTITY

Clothing		
T-shirt	799	50
Groceries		
Rice bag	2000	40
Furniture		
Office chair	4500	15
Books		
Python book	650	25
Electronics		
Headphones	1500	30

- Count number of products in each category.

CATEGORY_NAME	PRODUCT_COUNT
-----	-----
Clothing	1
Books	1
Furniture	1
Electronics	1
Groceries	1

- Find highest-priced product in each category.

CATEGORY_NAME	

PRODUCT_NAME	

HIGHEST_PRICE	

Clothing	
T-shirt	799
Groceries	
Rice bag	2000
Furniture	
Office chair	4500
Books	
Python book	650
Electronics	
Headphones	1500

- Display categories having more than 5 products.

```
SQL> SELECT c.Category_Name FROM Product p JOIN Category c ON p.Category_ID=c.Category_ID GROUP BY c.Category_Name HAVING COUNT(p.Product_ID)>5;
```

CATEGORY_NAME

Electronics

- Find average product price category-wise.

```
SQL> select c.category_name,Avg(p.Price) As Avg_price from product p JOIN Category c ON p.Category_id = c.Category_id GROUP BY c.Category_name;
```

CATEGORY_NAME	AVG_PRICE
-----	-----
Clothing	799
Books	650
Furniture	4500
Electronics	3183.33333
Groceries	2000